Framework Package Selection

Jens Bouma
Nicolas Llop Prieto
Husna Keskin
Ben Wierenga
Matz Leeflang

Version Control

Version	Date	Description
0.1	12-09-2024	Initial document
0.2	18-09-2024	Added overview of requirements
0.3	19-09-2024	Added Criterium of evaluation
0.4	20-09-2024	Added longlist
0.5	24-09-2024	Added shortlist
0.6	26-09-2024	Added Conclusion
0.7	04-10-2024	Changes after feedback
0.8	09-10-2024	Language control
0.9	10-10-2024	Document ready
1.0	05-01-2025	Ready for submission

Table of Contents

1. Introduction 1

2. Overview of Requirements 2

2.1. Ease of Learning 2

2.2. Database Support 2

2.3. Authentication and Authorization 2

2.4. Routing and URL Management 2

2.5. Form Handling and Validation 2

2.6. Frontend Integration 3

2.7. Scalability and Performance 3

2.8. Security Features 3

2.9. Deployment on Linux 3

2.10. Dependency Management 3

2.11. Testing and Debugging 4

2.12. Version Control Integration 4

3. Criterium for evaluation 5

3.1. Front-End Framework Criteria 6

3.1.1. Ease of Learning 6
3.1.2. Component-Based Architecture 6
3.1.3. State Management 6
3.1.4. Templating & Rendering 6
3.1.5. Performance 6
3.1.6. Integration with APIs 6
3.1.7. Tooling & Libraries 7
3.1.8. Testing & Debugging 7
3.1.9. Community & Support 7

3.2. Back-End Framework Criteria 7

3.2.1. Ease of Learning 7
3.2.2. Database Support 7
3.2.3. Compatible with Linux server 8
3.2.4. Authentication & Authorization 8
3.2.5. Routing & URL Management 8
3.2.6. Form Handling & Validation 8
3.2.7. Middleware & Extensibility 8
3.2.8. Scalability & Performance 8
3.2.9. Security Features 9
3.2.10. Community & Ecosystem 9
3.2.11. Testing & Debugging 9

4. Longlist 9

4.1. Front-end 9
4.1.1. React 9
4.1.2. Vue 10
4.1.3. Angular 10
4.1.4. Next.js 10
4.1.5. Svelte 11
4.1.6. Gatsby 11
4.2. Back-end 11
4.2.1. Express.js 11
4.2.2. Django 12
4.2.3. Flask 12
4.2.4. Laravel 12
4.2.5. Spring Boot 13
4.3. Selection 13
4.3.1. Front-end 13
4.3.2. Back-end 14
5. Shortlist 15
5.1. React.js 15
5.2. Next.js 16
5.3. Laravel 17
5.4. Django 18
5.5. Spring boot 19
6. Conclusion 20
6.1. Front-end 20
6.2. Back-end 20
Bibliography 22

1. Introduction

The Purpose of this document is to provide a detailed review and comparison of various software frameworks which are under consideration for the Keuzewijzer. The selection of an appropriate framework is a decision which significantly impacts the efficiency, scalability, architecture, performance and maintainability of the Keuzewijzer.

This document outlines the criteria for evaluation, the frameworks which are under consideration and the reason behind the framework which will be advised at the end of the document. By comparing various factors such as maintainability and efficiency but also what the learning curve is before a developer can completely understand the existing code.

The primary objectives of this framework selection process are:

- Ease of maintenance and further development
- Integrating smoothly with existing systems
- Providing strong foundation for long-term stability and updates

By following this approach, our goal is to select the most suitable web development framework that will enable the successful delivery of our project and future growth support.

2. Overview of Requirements

To ensure that the chosen framework meets the technical requirements of the Keuzewijzer, the following requirements have been defined:

2.1. Ease of Learning

The framework should be easy to learn, with clear and comprehensive documentation that guides developers through common tasks like setup, routing, database integration, and authentication. It's important that the framework has an active community, offering tutorials, examples, and resources for troubleshooting. A simple syntax and design pattern are key factors in ensuring a smooth learning curve, especially if you or your team are new to the language or framework.

2.2. Database Support

A good framework must support a variety of databases, including popular options like MySQL, PostgreSQL, and SQLite (the latter primarily for development purposes). It should have tools and libraries available which create an efficient environment to integrate with a database. The framework should also support database migrations for handling schema changes during the project's lifecycle.

2.3. Authentication and Authorization

The framework should come with or easily integrate user authentication mechanisms, allowing users to log in, register, and manage sessions securely.

2.4. Routing and URL Management

Efficient routing and URL management is key for directing users to different pages on the website, such as home, semester selection, or viewing their learning path. The framework should provide an easy way to define and manage routes. Support for RESTful APIs is an added advantage, allowing for data retrieval and manipulation through standard HTTP methods, especially if you plan to implement an API-based backend.

2.5. Form Handling and Validation

Handling forms is a key part of web applications, and the framework you choose should make it easier to create and manage them. Built-in form validation is important because it ensures user input is properly checked on the server side, which helps prevent mistakes or malicious data from being processed. It's also crucial to have a secure way to manage form submissions and user input to protect the integrity and security of the application.

2.6. Frontend Integration

The framework should allow for simple integration with modern frontend libraries or frameworks like React or Vue.js, giving you the ability to create more advanced, reactive interfaces.

2.7. Scalability and Performance

The framework needs to handle an increasing number of users as the website scales. It should be optimized for performance, ensuring that database queries, page loads, and data processing happen efficiently. The ability to manage multiple requests simultaneously and ensure smooth operation under load is vital, especially for educational websites with many concurrent users.

2.8. Security Features

Security should be a built-in consideration for any framework. It must provide protection against common web security vulnerabilities, such as Cross-Site Request Forgery (CSRF) and Cross-Site Scripting (XSS). Secure session management, HTTPS support, and user input validation are essential. Passwords should be stored securely with hashing algorithms, and sensitive data should be encrypted both in transit and at rest.

2.9. Deployment on Linux

Since the project will run on a Linux server, the framework must be compatible with Linux-based environments. It should have straightforward deployment processes on common Linux distributions and integrate well with web servers like Nginx or Apache. Support for containerization tools such as Docker or virtual environments is an added advantage, allowing for smooth deployment and maintenance.

2.10. Dependency Management

The framework should offer an easy way to manage and install dependencies, using tools like pip for Python-based frameworks or npm for JavaScript-based ones. As the project grows, it should allow for seamless installation of additional packages and libraries. Ideally, it should also support virtual environments or containerized solutions, helping to isolate dependencies and avoid conflicts between different projects.

2.11. Testing and Debugging

Built-in testing and debugging tools are a necessity for ensuring the quality of the project. The framework should support unit testing and provide tools for logging errors and tracking down bugs. Clear and actionable error messages, along with good support for debugging workflows.

2.12. Version Control Integration

The framework should easily integrate with version control systems like Git, enabling collaborative development and version tracking. This allows developers to track changes, revert to previous versions, and work concurrently on different features of the project. Git integration is crucial for teams working together, ensuring that everyone can contribute to the project efficiently while maintaining code quality.

3. Criterium for evaluation

Here we will set the criteria for the frameworks. These criteria are separated into different lists so that only the criteria that are applicable are used for front-end and not for back-end. These criteria are created from the listed requirements in *Overview of Requirements*. Each criterion also has a different weight to them. So, Ease of learning is more important than that the framework can run on a Linux server. That is the reason why we give each criterium a certain weight to it, meaning that a criterion with a higher weight is more important than one with a lower weight.

To describe the weights, we are using in this document a table that has been made with all the weights and an explanation to them. Because most frameworks have most of the criteria but on different levels, we will also use a partial score system. This means that when a framework scores MED on a criteria with a high weight it will score 2 points. And if it scores a LOW on any of the three criteria weights it will result in 0 points.

Weight	Score	Explanation
KO	Knock-out	This weighting is for criteria that are crucial for the high functionality of the application. If the framework does not meet a criterion with this weighting, its use for the application will be completely excluded.
High	4 points	Criteria with this weighting have a high priority. They are vital for developing a Minimum Viable Product (MVP). If the framework does not meet a criterion with this weighting, it will significantly hinder the development of the application.
Medium	2 points	The criteria at this level have a medium priority weighting. While they do not immediately obstruct the development of an MVP, they significantly contribute to the functionality and usability of the framework within the application.
Low	1 point	Criteria with this weighting have a lower priority. They are not crucial for the core functionality of the application but can still contribute to certain aspects of the framework's functionality or usability.

Table 1 Criterium weights

3.1. Front-End Framework Criteria

3.1.1. Ease of Learning

Description: The framework should be intuitive, with comprehensive documentation and active community support.
Weight: Medium
Rationale: While not immediately critical for the MVP, ease of learning improves developer onboarding and productivity in the long term.

3.1.2. Component-Based Architecture

Description: It should support modular development for reusable components and dynamic UIs.
Weight: High
Rationale: Essential for developing reusable components and dynamic UIs, crucial for building scalable MVPs.

3.1.3. State Management

Description: Must provide tools or libraries for managing state across components (e.g., Vuex, Redux).
Weight: High
Rationale: Vital for managing data across components efficiently, ensuring consistent application functionality.

3.1.4. Templating & Rendering

Description: Should offer efficient templating systems and support for server-side or client-side rendering.
Weight: High
Rationale: Necessary for ensuring efficient front-end performance and dynamic UI rendering, which directly impacts user experience.

3.1.5. Performance

Description: Needs to handle complex user interfaces with high performance, ensuring smooth interactivity.
Weight: High
Rationale: Core to the application's usability. Poor performance would make the application unusable and significantly degrade user experience.

3.1.6. Integration with APIs

Description: Should seamlessly integrate with back-end APIs (REST, GraphQL) to fetch or manipulate data.
Weight: High

Rationale: Critical for connecting with back-end services and enabling dynamic, data-driven functionality in the application.

3.1.7. Tooling & Libraries

Description: Should have a rich ecosystem of plugins and tools for routing, forms, and styling.
Weight: Medium
Rationale: Adds value to the development process by providing convenience, but not critical for MVP functionality.

3.1.8. Testing & Debugging

Description: Should include or support testing frameworks and provide efficient debugging tools.
Weight: Medium
Rationale: While essential for long-term maintenance and quality, basic testing and debugging can be done with external tools if needed for an MVP.

3.1.9. Community & Support

Description: An active user base and regular updates are essential to ensure long-term support.
Weight: Medium
Rationale: An active community contributes to the long-term viability of the framework but is not crucial for MVP creation.

3.2. Back-End Framework Criteria

3.2.1. Ease of Learning

Description: Should have a simple setup process with clear tutorials, enabling fast ramp-up for developers.
Weight: Medium
Rationale: Helpful for developer productivity and fast ramp-up, but not critical for MVP development.

3.2.2. Database Support

Description: Must support multiple database types (e.g., SQL, NoSQL) and provide ORM for database management.
Weight: High
Rationale: Essential for data persistence and management. A framework lacking robust database support would severely limit application functionality.

3.2.3. Compatible with Linux server

Description: Needs to be able to run on the existing Linux server.
Weight: medium
Rationale: Core requirement if the application needs to run on a Linux server; failing this would exclude the framework entirely.

3.2.4. Authentication & Authorization

Description: Needs to provide built-in authentication features (login, registration) and role-based access control.
Weight: High
Rationale: Vital for securing the application and managing users, necessary for any MVP dealing

3.2.5. Routing & URL Management

Description: Should make it easy to define routes and manage URLs, particularly for building RESTful APIs.
Weight: High
Rationale: Critical for defining endpoints and managing API routes, impacting how users and the system interact with the application.

3.2.6. Form Handling & Validation

Description: Should include features for handling forms and validating user input securely.
Weight: Medium
Rationale: Convenient for handling user input securely, but not necessary for initial MVP development, as workarounds exist.

3.2.7. Middleware & Extensibility

Description: Should allow for easy extension via middleware, enabling additional functionalities like logging and security.
Weight: Medium
Rationale: Adds flexibility to extend functionalities (logging, security), but not essential for MVP functionality.

3.2.8. Scalability & Performance

Description: Needs to handle a high number of requests efficiently and be scalable for growing traffic demands.
Weight: High
Rationale: Core to ensuring the back-end can handle increasing traffic and user demand efficiently, crucial for application growth.

3.2.9. Security Features

Description: Must provide built-in security features to guard against vulnerabilities (e.g., CSRF, XSS, SQL injection).
Weight: High
Rationale: Security is a non-negotiable aspect of any application, especially one handling sensitive data. Built-in security features are essential to protect against vulnerabilities.

3.2.10. Community & Ecosystem

Description: Should have strong community support, with extensive libraries and third-party integrations.
Weight: Medium
Rationale: Important for long-term support and flexibility, but not immediately critical for MVP functionality.

3.2.11. Testing & Debugging

Description: Should have built-in or easily integrable tools for unit testing and debugging back-end logic.
Weight: Medium
Rationale: Useful for ensuring code quality, but basic testing can be achieved through third-party tools in the MVP stage.

4. Longlist

The longlist of for the frameworks will consist of 3 lists, frameworks for front-end, for back-end and full-stack. Furthermore, is it important to acknowledge that there are more frameworks out there that could be used but we have chosen to make a list of the better and more used frameworks. We chose to do this do shorten the duration needed to research more frameworks and can start developing and prototyping earlier so that more feedback can be incorporated in the final product.

4.1. Front-end

4.1.1. React

React.js is an open-source JavaScript framework which is used for building dynamic user interfaces mostly for single-page applications (SPAs). It supports in the creation of reusable components, which uses their own logic, structure, and style, making the development process more efficient. Reacts Virtual DOM improves performance by only updating parts of the actual DOM when changes occur, leading to a faster and smoother UI rendering. The JSX(JavaScript xml)syntax makes it easy to write HTML elements directly in JavaScript which enhances code readability.

Additionally, React has a large variety of available tools and libraries for routing, state management, and server-side rendering, making it a great choice for building modern, scalable web applications.

4.1.2. Vue

Vue.js is a progressive JavaScript framework for building user interfaces, designed to be approachable and flexible. Vue is incrementally adaptable, meaning developers can use it to enhance parts of existing projects or build fully featured single-page applications (SPAs) from scratch. Vue's component-based architecture allows for the creation of reusable and modular UI elements, promoting code organization and maintainability.

Vue uses an HTML-based templating syntax that allows developers to declaratively bind the rendered DOM to the underlying data. With features like reactive data binding and computed properties, Vue makes it easy to manage dynamic data and create efficient, reactive UIs. It also offers seamless integration with tools and libraries for routing and state management, making it versatile enough for both small projects and large-scale applications.

4.1.3. Angular

Angular is a comprehensive, open-source web application framework maintained by Google, designed for building dynamic, single-page applications (SPAs). It provides a full-featured ecosystem that includes everything from data binding and dependency injection to routing and form handling, making it a powerful tool for developing complex, large-scale applications. Angular follows a component-based architecture, where the UI is divided into reusable, independent components that manage their own logic and views.

One of Agular's standout features is its use of two-way data binding, which automatically synchronizes the data between the model and the view, simplifying the management of dynamic content. It also utilizes TypeScript, a superset of JavaScript that adds static types, improving code reliability and scalability. Angular is ideal for enterprise-level applications due to its robust structure, extensive tooling, and strong community support.

4.1.4. Next.js

Next.js is a React-based open-source framework that enables server-side rendering (SSR) and static site generation (SSG) for building modern web applications. Developed by Vercel, it offers a highly optimized developer experience by providing features like file-based routing, automatic code splitting, and pre-rendering out of the box. These features help enhance the performance, scalability, and SEO-friendliness of React applications.

Next.js supports both server-side and client-side rendering, giving developers the flexibility to choose how their pages are rendered based on the specific needs of the application. It also integrates seamlessly with React's ecosystem, making it a go-to choice for developers looking to build fast, dynamic websites or progressive web apps. With its built-in API routes, Next.js can handle back-end logic as well, making it suitable for full-stack development.

4.1.5. Svelte

Svelte is a modern JavaScript framework designed for building highly efficient and lightweight web applications. Unlike traditional frameworks like React or Vue, Svelte shifts much of the work to the compile time. Instead of running in the browser, Svelte compiles components into pure, efficient JavaScript during the build process, resulting in faster performance and smaller bundle sizes.

With its reactive syntax, Svelte makes managing state and data updates simpler and more intuitive. The framework automatically tracks changes in your data and updates the DOM accordingly without the need for a virtual DOM. Svelte's simplicity and performance make it a great choice for developers seeking to build highly optimized, user-friendly applications with minimal overhead.

4.1.6. Gatsby

Gatsby is a React-based open-source framework used for building fast, static websites and applications. It combines the flexibility of React with powerful GraphQL data layer capabilities, allowing developers to pull data from various sources (like APIs, Markdown, or CMSs) and seamlessly integrate them into their websites. Gatsby's static site generation ensures that all pages are pre-rendered at build time, resulting in blazing-fast load speeds and improved SEO.

With its rich plugin ecosystem, Gatsby simplifies tasks like image optimization, progressive web app (PWA) support, and content management integration. It's ideal for creating blogs, marketing websites, and documentation sites, particularly for projects where performance, scalability, and SEO are critical concerns.

4.2. Back-end

4.2.1. Express.js

Express.js is a minimal and flexible web application framework for Node.js, designed to simplify the process of building server-side applications and APIs. As one of the most popular Node.js frameworks, Express provides a robust set of features for handling HTTP requests, routing, middleware integration, and more, making it an excellent choice for developing web servers and RESTful APIs.

Express.js offers a straightforward API for defining routes and middleware, allowing developers to quickly build and customize server-side logic. Its unopinionated nature means it doesn't enforce a specific structure, giving developers the freedom to organize their applications as they see fit. This flexibility, combined with a large ecosystem of plugins and middleware, makes Express a versatile and powerful tool for building scalable, high-performance web applications and services.

4.2.2. Django

Django is a high-level, open-source web framework for Python that emphasizes rapid development, clean design, and the principle of "don't repeat yourself" (DRY). It is known for its batteries-included approach, providing a comprehensive suite of built-in features such as an ORM (Object-Relational Mapping) for database interactions, an admin interface, authentication, and form handling.

Django's MTV (Model-Template-View) architecture separates data, user interface, and application logic, making it easier to manage and scale complex applications. Its robust security features, such as protection against common web vulnerabilities (e.g., SQL injection, cross-site scripting), and its strong community support make it a popular choice for building scalable, secure, and maintainable web applications quickly and efficiently.

4.2.3. Flask

Flask is a lightweight, open-source web framework for Python designed to be simple and flexible, making it ideal for small to medium-sized applications and microservices. Unlike more extensive frameworks like Django, Flask provides the essentials for web development while allowing developers the freedom to choose additional tools and libraries based on their needs.

Flask follows a minimalist approach, offering a basic core with features such as routing, request and response handling, and templating through Jinja2. Its simplicity allows developers to build applications quickly and with fewer constraints. Flask's modular design makes it easy to extend with various plugins for tasks like database integration, authentication, and more, giving developers the flexibility to tailor their applications as required.

4.2.4. Laravel

Laravel is a popular open-source PHP web framework designed for building modern, robust applications with an elegant syntax. It emphasizes simplicity, clarity, and developer productivity, offering a wide range of built-in tools for tasks like routing, authentication, caching, and database management. Laravel follows the MVC (Model-View-Controller) architecture, making it easy to organize and manage code.

One of Laravel's standout features is Eloquent, its powerful ORM (Object-Relational Mapping) system, which simplifies database interactions. Additionally, Laravel provides a Blade templating engine for clean, reusable front-end views, and a comprehensive ecosystem that includes tools like Laravel Forge for server management and Laravel Vapor for serverless deployment. This makes Laravel a great choice for both small projects and large, scalable applications.

4.2.5. Spring Boot

Spring Boot is a Java-based open-source framework designed to simplify the development of stand-alone, production-ready Spring applications. It builds on the Spring framework but removes much of the complexity by providing convention over configuration and opinionated defaults, allowing developers to get up and running quickly without needing to define extensive configurations.

Spring Boot comes with embedded web servers like Tomcat or Jetty, enabling applications to run independently without the need for external application servers. It supports RESTful API development, database integration, security, and microservices architecture, making it ideal for building scalable, enterprise-level applications. With features like auto-configuration and built-in dependency management, Spring Boot significantly reduces boilerplate code and improves development speed, making it a popular choice for Java developers.

4.3. Selection

To get the amount of points each framework will score according to the criteria we will use a Score-matrix.

4.3.1. Front-end

Citeria	React	Vue	Angular	Gatsby	Next	Svelte
1	MED	MED	LOW	HIGH	MED	HIGH
2	HIGH	HIGH	HIGH	MED	HIGH	MED
3	HIGH	MED	HIGH	MED	HIGH	MED
4	HIGH	HIGH	HIGH	MED	HIGH	HIGH
5	HIGH	HIGH	MED	HIGH	HIGH	HIGH
6	HIGH	HIGH	HIGH	MED	HIGH	HIGH
7	HIGH	MED	HIGH	MED	HIGH	LOW
8	HIGH	MED	HIGH	MED	HIGH	LOW
9	HIGH	MED	HIGH	HIGH	HIGH	MED
Total Score	28	25	24	19	28	19

Table 2 Front-end score-matrix

The shortlist for the front-end exist out of the 2 highest scoring frameworks: React.js and Next.js. Both frameworks scored 28 points from the criteria and are with that a

shared first place among the front-end frameworks. We are also going to work out this shortlist to see which of the frameworks is better for our project than the other.

4.3.2. Back-end

Citeria	Express.js	Django	Flask	Laravel	Spring Boot
1	MED	MED	MED	MED	LOW
2	HIGH	HIGH	MED	HIGH	HIGH
3	MED	MED	MED	MED	MED
4	MED	HIGH	MED	HIGH	HIGH
5	HIGH	HIGH	MED	HIGH	HIGH
6	LOW	MED	LOW	MED	MED
7	MED	LOW	LOW	MED	MED
8	HIGH	HIGH	MED	HIGH	HIGH
9	MED	HIGH	MED	HIGH	HIGH
10	MED	MED	MED	MED	MED
11	MED	MED	LOW	MED	MED
Total Score	28	31	19	32	31

Table 3 Back-end score matrix

the result of this list is that Laravel is slightly above Django and Spring Boot. We are also going to choose Laravel based on the results and earlier products, and skills within the current team.

5. Shortlist

Within this chapter we will run a deeper analysis on the two remaining frameworks: React.js and Next.js.

5.1. React.js

React is a popular JavaScript framework for building user interfaces, especially single-page applications (SPAs). It allows developers to build reusable components, which encapsulate logic, structure, and style, improving efficiency and maintainability.

Key Features:

- **Component-Based Architecture**: React's modular design encourages the reuse of UI components.
- **JSX (JavaScript XML)**: Allows developers to write HTML-like code within JavaScript, enhancing readability.
- **Virtual DOM**: React updates only the necessary parts of the real DOM, boosting performance.
- **Hooks**: Simplifies state management and side effects handling in functional components.

Ecosystem: React integrates well with other libraries for state management (e.g., Redux), routing (React Router), and form handling (Formik). Its ecosystem includes app development through **React Native**.

Strengths:

- **Flexibility**: Can be used for building small widgets or full applications.
- **Community**: Strong community support and job market presence.
- **Performance**: Efficient handling of dynamic UIs with its Virtual DOM.

Challenges:

- **Learning Curve**: Requires learning additional libraries for routing, state management, and form handling.
- **Setup Complexity**: Not an all-in-one solution, requiring integration with other tools.

React is a highly versatile and scalable library, ideal for modern, interactive applications across web and mobile platforms.

5.2. Next.js

Next.js is a React-based framework that extends React's capabilities by enabling server-side rendering (SSR), static site generation (SSG), and API routes. It's designed to build modern web applications with better performance, SEO, and scalability.

Key Features:

- **Server-Side Rendering (SSR)**: Pages can be rendered on the server, improving SEO and initial load performance.
- **Static Site Generation (SSG)**: Pre-renders pages at build time, making sites extremely fast.
- **File-Based Routing**: Automatically generates routes based on the file structure, simplifying navigation setup.
- **API Routes**: Built-in support for creating API endpoints within the same project.
- **Optimized Performance**: Code splitting, lazy loading, and automatic static optimization for faster load times.

Ecosystem: Next.js works seamlessly with React's ecosystem, including tools like React Router, Redux for state management, and Tailwind CSS for styling. It also supports serverless deployment on platforms like Vercel and AWS.

Strengths:

- **SEO Optimization**: Server-side rendering makes it ideal for SEO-critical applications.
- **Performance**: Static and server-side rendering ensures fast load times.
- **Full-Stack Capabilities**: With API routes, developers can build both front-end and back-end logic in the same project.
- **Easy Deployment**: Integrates well with serverless platforms for simple and scalable deployment.

Challenges:

- **Learning Curve**: Somewhat more complex than plain React due to SSR and routing features.
- **Build Time for Large Sites**: Static site generation can increase build times for large-scale projects.

Next.js is an excellent choice for React developers looking to improve performance, SEO, and scalability in web applications, offering full-stack capabilities and robust features for both small and large projects.

5.3. Laravel

Laravel is a popular PHP web framework designed for building modern, scalable web applications. It emphasizes simplicity, elegance, and developer productivity, offering a range of tools and built-in features that streamline common web development tasks such as routing, authentication, and database interactions.

Key Features:

- **Elegant Syntax**: Laravel is known for its readable and elegant syntax, making it accessible for developers familiar with PHP.
- **Blade Templating Engine**: Laravel's Blade is a powerful templating engine that allows for clean, reusable front-end views with simple logic.
- **Eloquent ORM**: The built-in Eloquent ORM (Object-Relational Mapping) simplifies database interactions by allowing developers to work with databases using an intuitive, object-oriented approach.
- **Comprehensive Ecosystem**: Laravel comes with various packages and tools, such as Laravel Forge (for server management) and Laravel Vapor (for serverless deployment), making it easier to handle deployment and scaling.
- **Authentication & Authorization**: Laravel has built-in support for user authentication, authorization, and security features like hashing and encryption.

Strengths:

- **Fast Development**: Laravel's powerful tools and out-of-the-box features reduce development time.
- **Community**: Laravel has a large, active community and extensive documentation, providing ample support for developers.
- **Flexible and Scalable**: Its extensive ecosystem makes Laravel suitable for projects of all sizes, from small apps to large enterprise solutions.

Challenges:

- **Performance**: Compared to some frameworks, Laravel can be slower in high-traffic applications due to its abstraction layers, though this can be mitigated with proper optimization.

Use Cases: Laravel is well-suited for building content management systems (CMS), e-commerce platforms, and API-driven applications.

5.4. Django

Django is a high-level Python web framework that promotes rapid development and clean, pragmatic design. It is often referred to as a batteries-included framework due to its comprehensive set of built-in tools and features that simplify common development tasks, such as authentication, database interaction, and URL routing.

Key Features:

- **MTV Architecture**: Django uses a Model-Template-View (MTV) architecture, which separates the database (Model), logic (View), and user interface (Template), making code easier to manage and scale.
- **ORM and Database Migrations**: Django's ORM makes database operations intuitive and integrates seamlessly with various databases. Its built-in migration system handles database schema changes automatically.
- **Admin Interface**: Django includes a powerful, customizable admin interface for managing content, making it easier to create and manage web applications with minimal additional code.
- **Security**: Django is known for its robust security features, such as protection against SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).

Strengths:

- **Rapid Development**: Django's extensive features allow developers to focus on writing application logic without reinventing the wheel, speeding up the development process.
- **Security**: Django has built-in security mechanisms that automatically mitigate many common vulnerabilities.
- **Scalability**: Django is suitable for both small projects and large-scale web applications with millions of users.

Challenges:

- **Complexity for Small Projects**: For smaller applications, Django can feel heavy due to its comprehensive feature set.
- **Performance**: Django's abstraction layers may introduce performance overhead, though optimization techniques are available.

Use Cases: Django is an excellent choice for building content-heavy sites, social media platforms, and data-driven applications.

5.5. Spring boot

Spring Boot is a Java-based framework designed to simplify the development of production-ready, stand-alone applications. It builds on the traditional Spring Framework by providing convention over configuration, which reduces boilerplate code and allows developers to create and deploy applications more quickly.

Key Features:

- **Microservices-Friendly**: Spring Boot is well-suited for microservices architectures, allowing developers to build independently deployable services that are easy to scale and manage.
- **Embedded Servers**: Spring Boot applications come with embedded web servers (like Tomcat or Jetty), so there's no need for a separate application server. This simplifies deployment and makes applications easier to run.
- **Spring Ecosystem**: Spring Boot integrates seamlessly with the broader Spring ecosystem, including tools for dependency injection, database access, security, and testing.
- **Configuration Management**: Spring Boot's auto-configuration and externalized configuration make it easier to manage complex applications, as developers can control configurations using properties files or environment variables.

Strengths:

- **Rapid Setup**: Spring Boot removes much of the configuration overhead from traditional Spring applications, making it easier to start new projects.
- **Scalability**: Built on Java, Spring Boot is highly scalable and suitable for enterprise-level applications with complex business logic.
- **Robust Security**: Spring Security, a part of the Spring ecosystem, provides robust security features, including authentication, authorization, and protection against common attacks.

Challenges:

- **Learning Curve**: Spring Boot can be difficult to learn for developers new to Java or the Spring ecosystem due to its rich feature set.
- **Resource-Intensive**: Java applications, including Spring Boot, tend to use more memory and resources compared to frameworks built with lighter languages like Python or PHP.

Use Cases: Spring Boot is ideal for enterprise applications, microservices, and APIs that require scalability, security, and maintainability.

6. Conclusion

6.1. Front-end

While React.js is a great choice for building component-based user interfaces, Next.js offers a more comprehensive solution by extending React's core capabilities with features like server-side rendering (SSR), static site generation (SSG. These features are of great importance for applications where performance and scalability are important.

Reasons to Choose Next.js:

Performance: Next.js optimizes page load times by pre-rendering pages either at build time (SSG) or on each request (SSR), significantly improving both initial load times

Simplified Routing: Next.js simplifies routing with its file-based routing system, reducing the need for additional routing libraries like React Router.

When to Choose React: If your project requires only client-side rendering, or you're building a single-page application without the need for server-side optimizations, React.js might be the better choice. It offers more flexibility in terms of setup, but often requires additional tools and configurations for routing, state management, and server-side logic.

Verdict:

For most modern web applications, especially those requiring high performance and SEO optimizations, Next.js is the best choice. It builds on React's strengths while adding critical features for a broader range of use cases, making it a more powerful and complete solution for full-stack and performance-focused applications. Furthermore, with the prototype already being made in next.js it improves the development time for the Keuzewijzer.

6.2. Back-end

When deciding between Laravel, Django, and Spring Boot for the Keuzewijzer project, the best option will depend on factors such as ease of learning, development speed, and the complexity of the system. Since Keuzewijzer is a tool for managing student semester selections, the ideal framework should offer efficient handling of data, user management, and flexibility to grow as the project evolves. Each framework has its strengths, but the most suitable one will provide a balance between simplicity and scalability, making development smoother without unnecessary complexity.

Reasons to Choose Laravel:

Laravel is well-suited for the Keuzewijzer project because it provides a balance of simplicity and power, making it ideal for rapid development. Laravel's elegant syntax

and built-in features like Blade templating and Eloquent ORM make it easy to manage semester selections and student data. The authentication system is robust, supporting user roles, which is important for differentiating between student and admin users.

Laravel’s extensive documentation and beginner-friendly learning curve allow for a quick development process, especially for developers new to back-end frameworks.

Reasons to Choose Django:

Django excels in handling complex web applications and provides a more structured approach with its Model-Template-View (MTV) architecture. Django’s built-in admin interface would simplify managing semesters and student information, and its strong security features offer excellent protection for sensitive data.

While Django may have a steeper learning curve than Laravel, especially if you're new to Python, it offers a comprehensive set of tools that make it easy to scale as the project grows. Django's ORM and automatic database migrations make it a solid choice for managing the Keuzewijzer's database.

Reasons to Choose Spring Boot:

Spring Boot is a powerful framework, but it’s better suited for enterprise-level or highly complex applications. It provides excellent tools for building scalable microservices, which might be overkill for a project like Keuzewijzer. Security is a key strength of Spring Boot, with Spring Security offering extensive authentication and authorization features. However, the learning curve for Spring Boot is steeper, and the Java-based setup may be more resource-intensive.

While Spring Boot excels in scalability and performance, it may be more than what’s needed for a system like Keuzewijzer, unless the project is expected to grow into a large, complex platform.

Verdict:

For the Keuzewijzer project, Laravel is the best choice due to its ease of learning, fast development, and built-in tools for managing forms, authentication, and databases. Laravel’s simplicity makes it ideal for quickly building and deploying the Keuzewijzer, while its flexibility allows for future growth.

If you prefer working with Python or need more built-in features for scalability and security, Django would be a strong alternative. It’s especially useful if you want a ready-made admin interface and strong security out of the box.

Spring Boot however, is likely too complex for a project of this scope, making it a better fit for large-scale, enterprise-grade applications rather than a student semester choice wiser system.

Bibliography

Django. (sd). *Django documentation*. Opgehaald van DjangoProject: https://docs.djangoproject.com/en/5.1/

Express. (sd). *express*. Opgehaald van expressjs: https://expressjs.com/

Flask. (sd). *Welcome to flask - flask documentation*. Opgehaald van flask palletsprojects: https://flask.palletsprojects.com/en/3.0.x/

gatsby. (sd). *Welcome to the Gatsby Way of Building*. Opgehaald van Gatsby: https://www.gatsbyjs.com/docs/

installation - laravel 11.x. (sd). Opgehaald van Laravel: https://laravel.com/docs/11.x

nextjs. (sd). *Docs | nextjs*. Opgehaald van nextjs: https://nextjs.org/docs

React. (sd). *Quick start*. Opgehaald van react: https://react.dev/learn

Robinson, L. (2022). *State of javascript 2022: Front-end frameworks*. Opgehaald van stateofjs: https://2022.stateofjs.com/en-US/libraries/front-end-frameworks/

Spring. (sd). *Spring Boot*. Opgehaald van Spring: https://spring.io/projects/spring-boot

Svelte. (sd). *introduction*. Opgehaald van svelte: https://svelte.dev/docs/introduction

Vercel. (2024, october 1). *Next.js on vercel*. Opgehaald van vercel: https://vercel.com/docs/frameworks/nextjs

vue. (sd). *Introduction*. Opgehaald van Vue: https://vuejs.org/guide/introduction.html